

TRABALHO PRÁTICO 2 - COMPUTAÇÃO GRÁFICA

Aluna: Yasmin Viegas - 800989

Professora: Rosilane Mota

Link para vídeo no google drive:  TP2_CG_Yasmin.mp4

Como executar o trabalho:

Primeiro se deve extrair o conteúdo do TP2_CG_Yasmin.zip

Depois executar **.exe** em:

C:\TP2_CG_Yasmin\Paint-Comp.Grafic\dist\ComputacaoGrafica\ComputacaoGrafica.exe

Ou se preferir rodar o Python:

- Requisitos: Python 3.8 ou superior com pip disponível
- Entrar na pasta \Paint-Comp.Grafic\ (onde está o código) e dentro dela:
 - Instalar dependências: `pip install -r requirements.txt`
 - Executar: `python main.py`

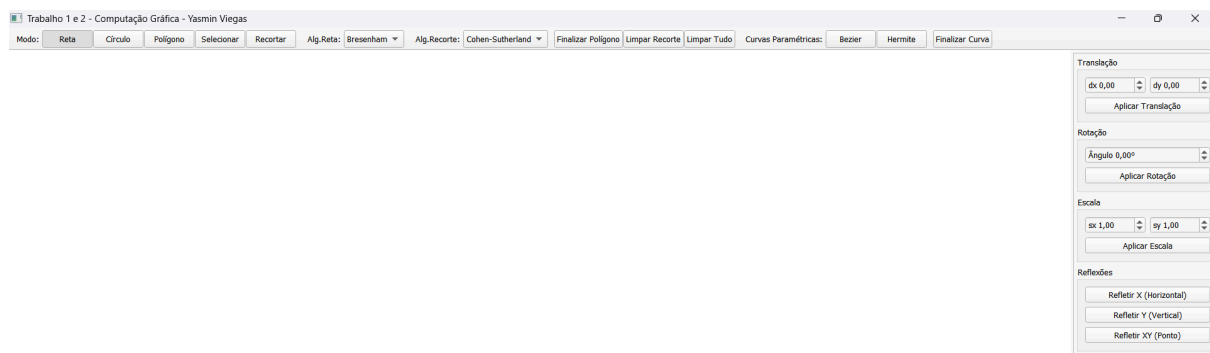
Códigos também disponíveis em: <https://github.com/yasyaaaas/Paint-Comp.Grafic/tree/main>

Descrição do trabalho

Este trabalho estende o TP1, reaproveitando toda a estrutura já implementada (rasterização, recorte, transformações geométricas e interface) e adicionando suporte a duas curvas paramétricas: **Bézier** e **Hermite**. A lógica das curvas foi organizada em uma pasta separada chamada **curvas/**, mantendo o restante do projeto intacto.

Interface

A interface ganhou uma segunda barra de ferramentas para as curvas paramétricas, separada visualmente da toolbar principal do TP1. Nela estão os botões de modo **Bézier** e **Hermite**, além do botão **Finalizar Curva**. O uso é análogo ao Polígono: o usuário clica nos pontos de controle um a um e finaliza ao clicar no botão ou com duplo clique.



Arquivos novos

curvas/bezier.py

Implementa a curva de Bézier utilizando os polinômios de Bernstein para calcular os pontos da curva a partir de uma lista de pontos de controle. A classe **CurvaBezier** recebe como parâmetro o callback de rasterização de retas já existente no projeto, de forma que a curva respeita o algoritmo de reta escolhido pelo usuário (Bresenham ou DDA) na toolbar.

Os pontos da curva são calculados pela função **bezier(t, pontos)**, que avalia o somatório de Bernstein para um dado parâmetro $t \in [0, 1]$:

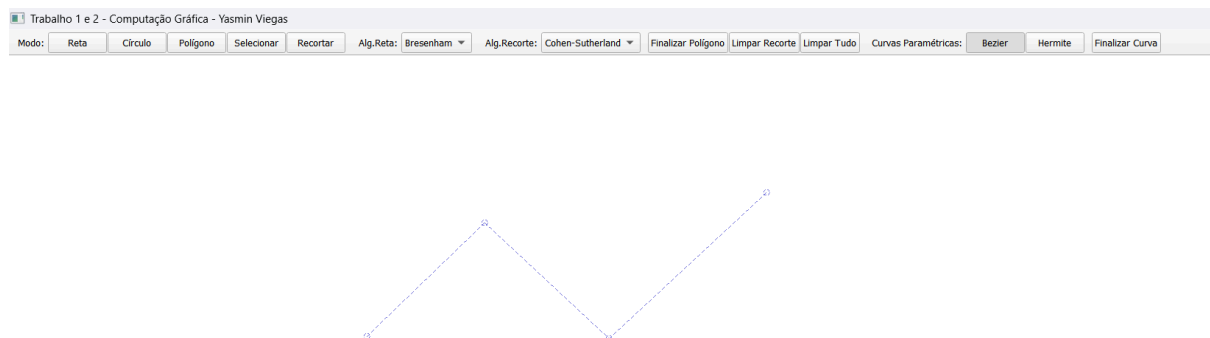
$$B(t) = \sum B_{i,n}(t) \cdot P_i$$

onde $B_{i,n}(t) = C(n,i) \cdot t^i \cdot (1-t)^{(n-i)}$ é o i -ésimo polinômio de Bernstein de grau n . A função **pontos_bezier** mostra a curva em **num_passos** valores uniformes de t , gerando a lista de pontos que são então conectados por segmentos de reta pelo rasterizador.

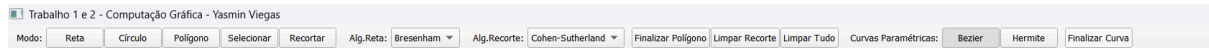
OBS: a curva interpola apenas os pontos extremos (P_0 e P_n). Os pontos intermediários são de controle, assim atraem a curva sem serem tocados por ela.

Referência do código: Gist "Draw Bezier curves using Python and PyQt" - disponível em <https://gist.github.com/1274149/ca37e497b3f2a16c9d3ec4889ed63c80986e9dba>

Usuário coloca os pontos



Curva formada depois de clicar no botão **Finalizar Curva**, ou duplo clique no último ponto



curvas/hermite.py

Implementa a curva de **Hermite** cúbica, que interpola dois pontos extremos (P0 e P1) e utiliza dois pontos de controle (R0 e R1) para definir as tangentes nos extremos. A curva é calculada a partir das funções de base de Hermite, que são combinações polinomiais cúbicas.

Como usar: O usuário deve clicar na tela na ordem: P0 (início), R0 (controle da tangente em P0), R1 (controle da tangente em P1) e P1 (fim). A curva é finalizada com o botão "Finalizar Curva" ou duplo clique.

Para cada parâmetro $t \in [0, 1]$, a posição é dada por:

$$H(t) = h_{00}(t) \cdot P_0 + h_{10}(t) \cdot T_0 + h_{01}(t) \cdot P_1 + h_{11}(t) \cdot T_1$$

onde:

- $h_{00}(t) = 2t^3 - 3t^2 + 1$
- $h_{10}(t) = t^3 - 2t^2 + t$
- $h_{01}(t) = -2t^3 + 3t^2$
- $h_{11}(t) = t^3 - t^2$

A curva de Hermite é definida por **4 pontos de controle**:

- **P0** – ponto inicial (a curva começa exatamente nele).
- **R0** – ponto auxiliar que define a tangente em P0: $T_0 = R_0 - P_0$.
- **R1** – ponto auxiliar que define a tangente em P1: $T_1 = P_1 - R_1$.
- **P1** – ponto final (a curva termina exatamente nele).

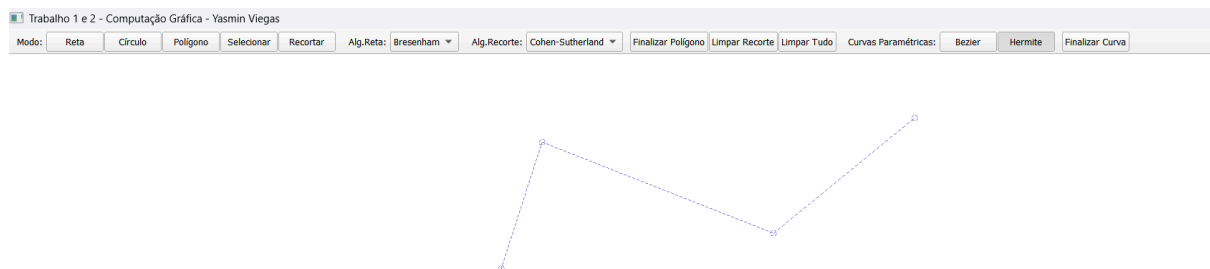
$T_0 = R_0 - P_0$ e $T_1 = P_1 - R_1$. A classe **CurvaHermite** recebe o callback de rasterização, de modo que a curva é desenhada por segmentos de reta, respeitando o algoritmo de reta escolhido (Bresenham ou DDA).

A curva resultante **interpola P0 e P1** (passa exatamente por eles) e a forma é controlada por R0 e R1. Quanto mais distante R0 estiver de P0, mais "esticada" será a tangente, e o mesmo para R1.

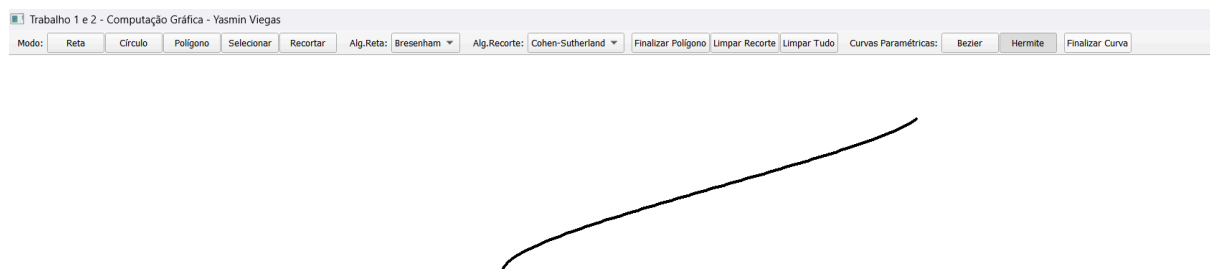
OBS: A curva interpola P0 e P1, mas não toca em R0 nem R1; esses pontos apenas controlam a direção e intensidade das tangentes.

Referência do código: Adaptado do repositório - <https://github.com/vedantyadu/Hermite-cubic-spline>

Usuário coloca os pontos



Curva formada depois de clicar no botão **Finalizar Curva**, ou duplo clique no último ponto



Arquivos modificados

main.py

Ganhou uma parte da toolbar exclusiva para as curvas paramétricas. Nela estão os botões de modo Bézier e Hermite e o botão Finalizar Curva. O método `_set_modulo` foi atualizado para limpar os pontos de controle em construção ao trocar de modo.

paint.py

Passou a importar **CurvaBezier** e **CurvaHermite** da pasta **curvas/** e instancia os dois renderizadores no construtor, passando **_rasterizar_reta** como callback. Foram acrescentados:

- **self.pontos_curva** - lista temporária de pontos clicados durante a construção da curva
- **self._curvas_bezier** e **self._curvas_hermite** - listas persistentes de curvas finalizadas, cada uma armazenando os pontos de controle utilizados
- **_press_curva(x, y)** - handler de mouse que adiciona cada clique à lista temporária
- **finalizar_curva()** - salva a curva na lista correspondente e aciona o redesenho
- Preview visual no **paintEvent** mostrando os pontos e o polígono de controle tracejado enquanto a curva ainda está sendo construída

O **limpar_tela** também foi atualizado para resetar as listas de curvas.

Como usar as curvas

- Selecionar modo **Bézier** ou **Hermite** na toolbar
- **Bézier**: clicar na tela para adicionar os pontos de controle (mínimo 2).
- **Hermite**: clicar exatamente **4 pontos** na ordem: P0 (início), R0 (controle da tangente em P0), R1 (controle da tangente em P1) e P1 (fim).
- Clicar em **Finalizar Curva** ou dar **duplo clique** para rasterizar e salvar a curva
- Durante a construção, os pontos clicados aparecem marcados com círculos e conectados por uma linha tracejada azul, formando o polígono de controle visível
- É possível desenhar quantas curvas quiser, alternando cores e modos livremente